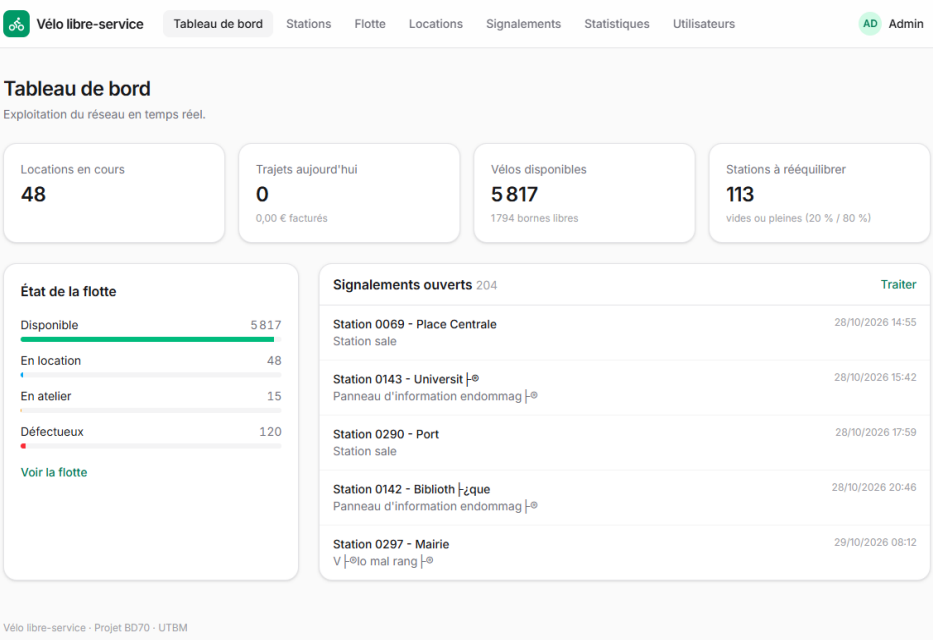


UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Création et optimisation d'un système d'informations de gestion d'un réseau de vélos en libre-service

Rapport de projet BD70 - A26

AUBERT Colin - GRATIA Alban - PREVOST Antoine

Département Génie informatique
3A FISA

Mots clefs

Base de données - MariaDB - InnoDB - MySQL - Vélo - Optimisation - Automatisation

AUBERT Colin - GRATIA Alban - PREVOST Antoine **Rapport de projet BD70 - A26**

Résumé

Ce rapport présente la conception et la réalisation d'un système d'information pour un réseau de vélos en libre-service : une base de données relationnelle MariaDB en troisième forme normale, dont les règles métier (un vélo à un seul endroit, une borne par vélo, une location à la fois par usager, calcul automatique de la durée et du coût) sont garanties par des contraintes, des déclencheurs et des procédures stockées, et une application web PHP (Slim 4, Twig) qui l'exploite selon trois rôles : usager, opérateur et administrateur. La base a été peuplée avec un jeu de données factices de près de 5 Go (5 millions de locations, 10 millions d'événements historisés). À ce volume, plusieurs statistiques ne répondaient plus en un temps raisonnable : une démarche de mesure, d'analyse des plans d'exécution et de réécriture des requêtes a ramené chaque onglet de la page de statistiques sous 5 secondes sur la période par défaut de 30 jours (de 2,5 à 3,4 s sans cache, moins de 0,1 s avec), avec des gains allant jusqu'à un facteur 70 sur la requête la plus coûteuse. Le rapport détaille ces évolutions et les mesures qui les justifient.

utbm
université de technologie
Belfort-Montbéliard

Table des matières

1 Introduction **5**

1.1 Contexte et objectifs 5

1.2 Organisation et outils 5

1.3 Couverture du cahier des charges 5

2 Analyse des besoins **6**

2.1 Acteurs 6

2.2 Règles métier 8

3 Conception de la base de données **8**

3.1 Modèle conceptuel 8

3.2 Modèle logique 8

3.3 Normalisation 11

3.4 Contraintes d'intégrité 12

3.5 Index 12

4 Implémentation SQL **13**

4.1 Déclencheurs et historisation 13

4.2 Procédures et fonctions métier 14

4.3 Événements planifiés 15

4.4 Sécurité 15

5 Jeu de données factices **15**

5.1 Génération 15

5.2 Réalisme des données 16

5.3 Simulation de l'activité 16

5.4 Volumes 17

6 Application web **18**

6.1 Architecture 18

6.2 Fonctionnalités par rôle 19

6.3 Page de statistiques 19

7 Évolution du projet **20**

7.1 Chronologie 20

7.2 D'un prototype PHP natif à une application structurée 20

7.3 Problèmes rencontrés et solutions 21

8 Analyse des performances **21**

8.1 Protocole de mesure 21

8.2 Constat initial 22

8.3 Configuration du serveur 22

8.4 Réécriture des requêtes 23

8.4.1 Trajets origine-destination 23

8.4.2 Indisponibilité de la flotte 24

8.4.3 Remplissage des stations 24

8.4.4 Index couvrants 24

8.5 Résultats 25

8.5.1 Temps des requêtes principales 25

8.5.2 Montée en charge 26

8.5.3 Temps d'affichage et cache applicatif 26

8.6 Limites et pistes d'amélioration 27

9 Conclusion **27**

A Installation **28**

B Reproduire les mesures **28**

1 Introduction

1.1 Contexte et objectifs

Le projet de l'UV BD70 consiste à concevoir et développer le système d'information d'un réseau de vélos en libre-service à l'échelle d'une agglomération. Le système gère les stations et leurs bornes, la flotte de vélos mécaniques et électriques, les usagers et leurs abonnements, les locations et leur facturation, la maintenance et la régulation du réseau. Il doit fournir à l'exploitant une vision précise de l'utilisation du réseau (trajets, heures de pointe, stations saturées ou vides) afin d'optimiser le rééquilibrage des vélos.

Le sujet insiste sur deux points, qui ont guidé nos choix :

- la **cohérence des données** : un vélo ne peut être qu'à un seul endroit à la fois, une borne n'accueille qu'un vélo, un usager n'emprunte qu'un vélo à la fois ;
- l'**automatisation des calculs** dans la base : durée et coût des locations, historisation des états et des positions, facturation et expiration des abonnements.

1.2 Organisation et outils

- **SGBD** : MariaDB 10.4 (distribution XAMPP), moteur InnoDB ; scripts SQL versionnés et ré-exécutables (`schema.sql`, `triggers.sql`, `procedures.sql`, `seed.sql`, `simulation.sql`).
- **Application** : PHP 8 avec le micro-framework Slim 4, gabarits Twig, Tailwind CSS et Chart.js pour les graphiques.
- **Gestion de projet** : dépôt Git commun, une branche de développement (`dev`), une branche de documentation (`docs`) et des branches de fonctionnalités.

Le travail a été réparti par domaine, chaque membre prenant en charge une partie du livrable :

- **Colin Aubert** : premier prototype de l'interface (pages de base, navigation, transitions entre pages, thème sombre) ;
- **Alban Gratia** : diagramme entité-association et première optimisation de la page de statistiques (découpage en onglets, pagination, cache applicatif) ;
- **Antoine Prévost** : modèles de données, scripts SQL (schéma, déclencheurs, procédures, génération des données), application Slim et analyse des performances sur la base complète.

Chaque évolution est développée sur une branche dédiée puis reportée sur `dev` ; le rapport et les diagrammes vivent sur la branche `docs`. Les scripts SQL étant versionnés et ré-exécutables, chacun peut reconstruire une base identique sur son poste à partir du dépôt.

1.3 Couverture du cahier des charges

Le tableau 1 met en regard chaque exigence du sujet et sa réalisation ; les parties suivantes en détaillent la conception.

TABLE 1 – Exigences du sujet et réalisation

Exigence	Réalisation	Interface
Inscription, connexion, rôles	tables `utilisateur`, `usager`, `operateur`, `administrateur` ; procédures `sp_inscrire_usager`, `sp_authentifier` ; contrôle d'accès par rôle	oui
Stations et bornes	tables `station`, `borne` ; vue `v_stations_occupation` (vélos disponibles par type, bornes libres, en temps réel) ; création, modification, suppression, recherche et filtres	oui
Flotte	table `velo` (type, mise en service, état, batterie, borne) ; contraintes de position et de batterie ; historiques d'état et de position alimentés par déclencheurs	oui
Abonnements et tarification	tables `formule`, `tranche_tarifaire`, `abonnement`, `facture` ; fonction `fn_cout_formule` ; souscription, renouvellement, résiliation ; expiration par événement planifié	oui
Locations	procédures `sp_emprunter_velo`, `sp_rendre_velo` ; durée calculée par la base, coût calculé par déclencheur ; unicité « une location en cours » par vélo et par abonnement	oui
Signalements et maintenance	tables `signalement`, `intervention` ; file de traitement des signalements ; procédures d'intervention	partielle
Régulation	procédures `sp_stations_a_reequilibrer`, `sp_proposer_deplacements` ; tables `vehicule`, `tournee`, `ligne_tournee`	partielle
Statistiques et rapports	six procédures `sp_stat_*` ; page à onglets, graphiques, export CSV	oui
Ajout, modification, suppression, recherche, filtres	stations, comptes, profil ; filtres sur la flotte et les locations ; pagination	oui
3NF	voir partie 3 (dénormalisations contrôlées justifiées)	–
Données factices en quantité	procédures `sp_seed_*` ; environ 5 Go (partie 5)	–
Sécurité	compte applicatif à droits minimaux (jusqu'à la colonne), mots de passe bcrypt, jetons CSRF, requêtes préparées	–
Rapport de performances	partie 8	–

La maintenance et la régulation ne sont pas exigées dans l'interface par le sujet : elles sont entièrement réalisées dans la base (tables, procédures, données factices), et l'interface en expose une partie (file des signalements, propositions de rééquilibrage).

2 Analyse des besoins

2.1 Acteurs

- **Visiteur** : consulte la disponibilité du réseau, s'inscrit, se connecte.
- **Usager** : souscrit et gère son abonnement, emprunte et restitue des vélos, consulte ses trajets et ses factures, signale un problème.
- **Opérateur** (régulation ou maintenance) : suit la flotte et les locations, traite les signalements, consulte les statistiques et les propositions de rééquilibrage.

– **Administrateur** : droits de l'opérateur, plus la gestion des stations, des bornes et des comptes.

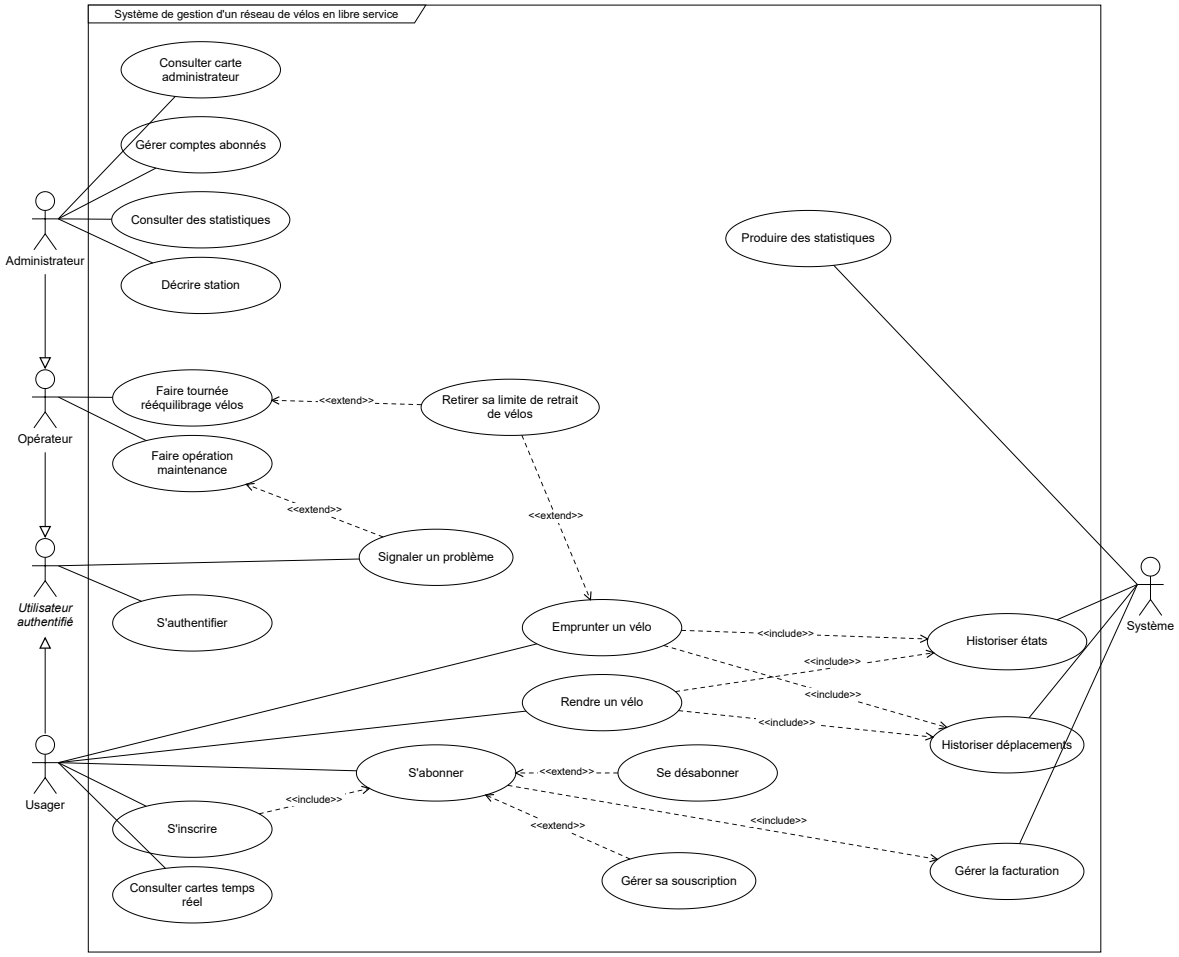


FIGURE 1 – Diagramme de cas d'utilisation

Le diagramme (figure 1) organise les acteurs en hiérarchie : l'**utilisateur authentifié** est l'acteur générique (s'authentifier, signaler un problème) ; l'**usager** et l'**opérateur** en héritent, et l'**administrateur** est un opérateur doté de droits supplémentaires (comptes, description des stations). Dans l'application, un visiteur non connecté peut en outre consulter la disponibilité du réseau et s'inscrire.

Les cas d'utilisation se regroupent en trois familles :

– **usage du réseau** (usager) : s'abonner, gérer sa souscription, se désabonner, emprunter et rendre un vélo ;
– **exploitation** (opérateur) : tournées de rééquilibrage, opérations de maintenance, consultation des statistiques ;
– **administration** : gestion des comptes et description des stations.

Un acteur secondaire, le **système**, représente les traitements automatiques de la base. Les relations « include » montrent qu'emprunter ou rendre un vélo déclenche toujours l'historisation de son état et de sa position, et que l'abonnement entraîne la facturation ; les relations « extend » marquent les cas facultatifs, comme la gestion ou la résiliation d'une souscription existante. Ces inclusions correspondent directement aux déclencheurs et aux événements planifiés de la partie 4.

2.2 Règles métier

Les règles suivantes, issues du sujet, sont garanties par la base elle-même et non par l'application (partie 4) :

1. un vélo ne peut être qu'à un seul endroit à la fois (sur une borne, en location ou en atelier) ;
2. une borne n'accueille qu'un seul vélo ;
3. un usager n'emprunte qu'un vélo à la fois, et seulement avec un abonnement valide ;
4. un vélo signalé défectueux ou en atelier ne peut pas être emprunté ;
5. la durée et le coût d'une location sont calculés automatiquement à la restitution ;
6. tout changement d'état ou de position d'un vélo est historisé.

3 Conception de la base de données

3.1 Modèle conceptuel

Le modèle entité-association (figure 2, en notation de Chen) s'articule autour de quatre ensembles :

- **le réseau** : une station possède des bornes, entité faible identifiée par son numéro au sein de la station ; les relevés de remplissage sont eux aussi rattachés à leur station ;
- **la flotte** : un vélo est *stationné* sur au plus une borne, et une borne accueille au plus un vélo (cardinalités 0,1 – 0,1) ; ses changements d'état et de position sont conservés dans deux historiques ;
- **la clientèle** : un usager souscrit des abonnements à une formule, dont le prix au trajet est décrit par des tranches tarifaires (entité faible de la formule) ; les factures sont émises par abonnement ;
- **l'exploitation** : les signalements visent soit un vélo, soit une station ; une intervention d'un opérateur de maintenance retire un vélo du réseau ; une tournée de régulation, réalisée avec un véhicule, déplace des vélos d'une station à l'autre (lignes de tournée).

La **location** est l'association centrale : elle relie un abonnement, un vélo, une borne de départ et, une fois le vélo rendu, une borne d'arrivée. Elle porte les dates d'emprunt et de restitution, d'où sont dérivés la durée et le coût. Rattacher la location à l'abonnement plutôt qu'à l'usager permet d'appliquer la tarification en vigueur au moment du trajet et de regrouper les trajets sur les factures de l'abonnement.

Les utilisateurs sont modélisés par **spécialisation** : tout utilisateur (identité, email, mot de passe) est soit un usager, soit un opérateur, et un administrateur est un opérateur. Ce choix reflète directement les rôles de l'application : un administrateur dispose de tous les droits d'un opérateur.

3.2 Modèle logique

Le schéma relationnel compte vingt tables (figure 3). En notation textuelle, la clé primaire est soulignée et les clés étrangères sont précédées de #.

Utilisateur (id_utilisateur, nom, prénom, email, mot_de_passe_hash, date_inscription)

Diagramme entité-association — Réseau de vélos en libre-service

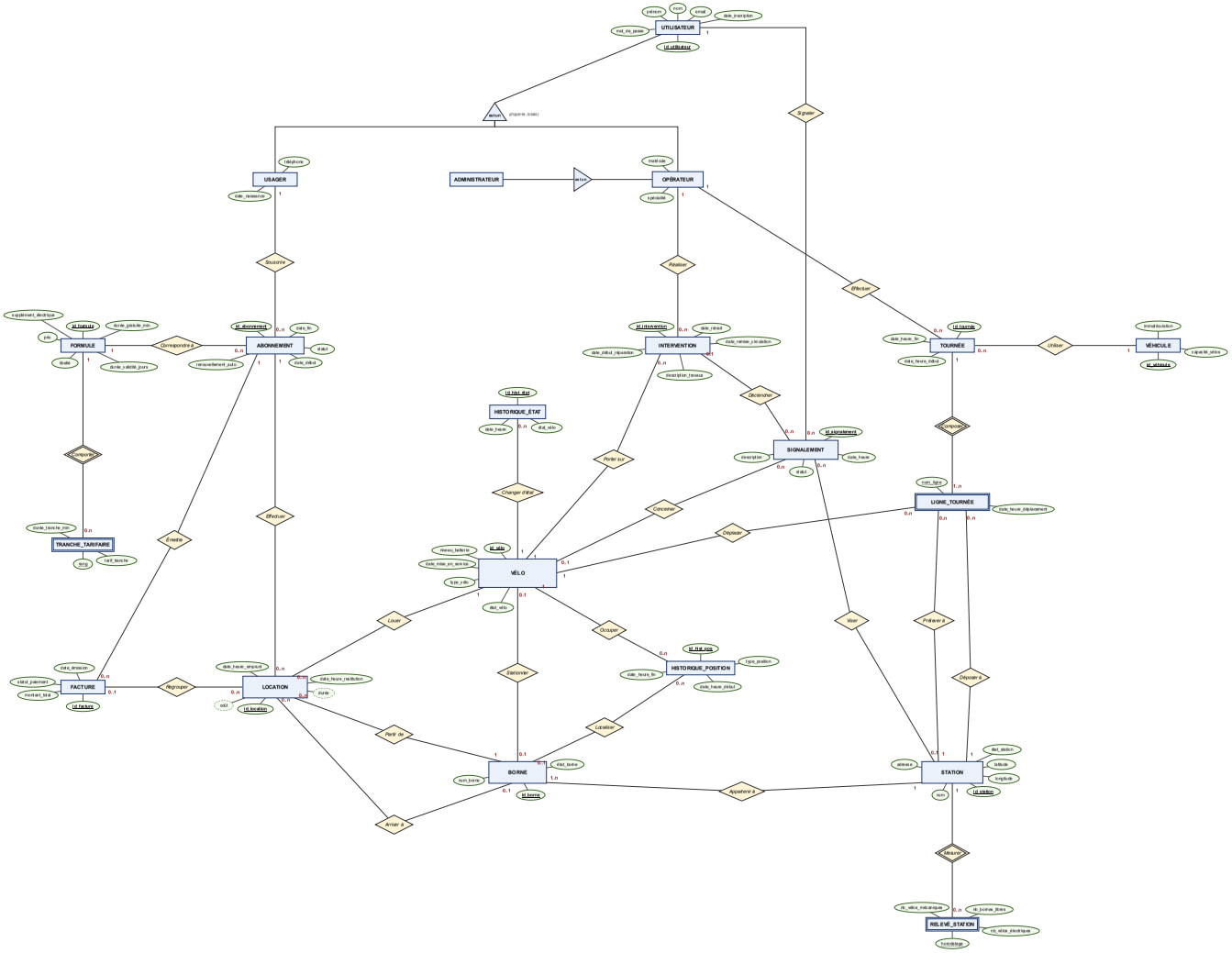


FIGURE 2 – Schéma entité-association (notation de Chen)

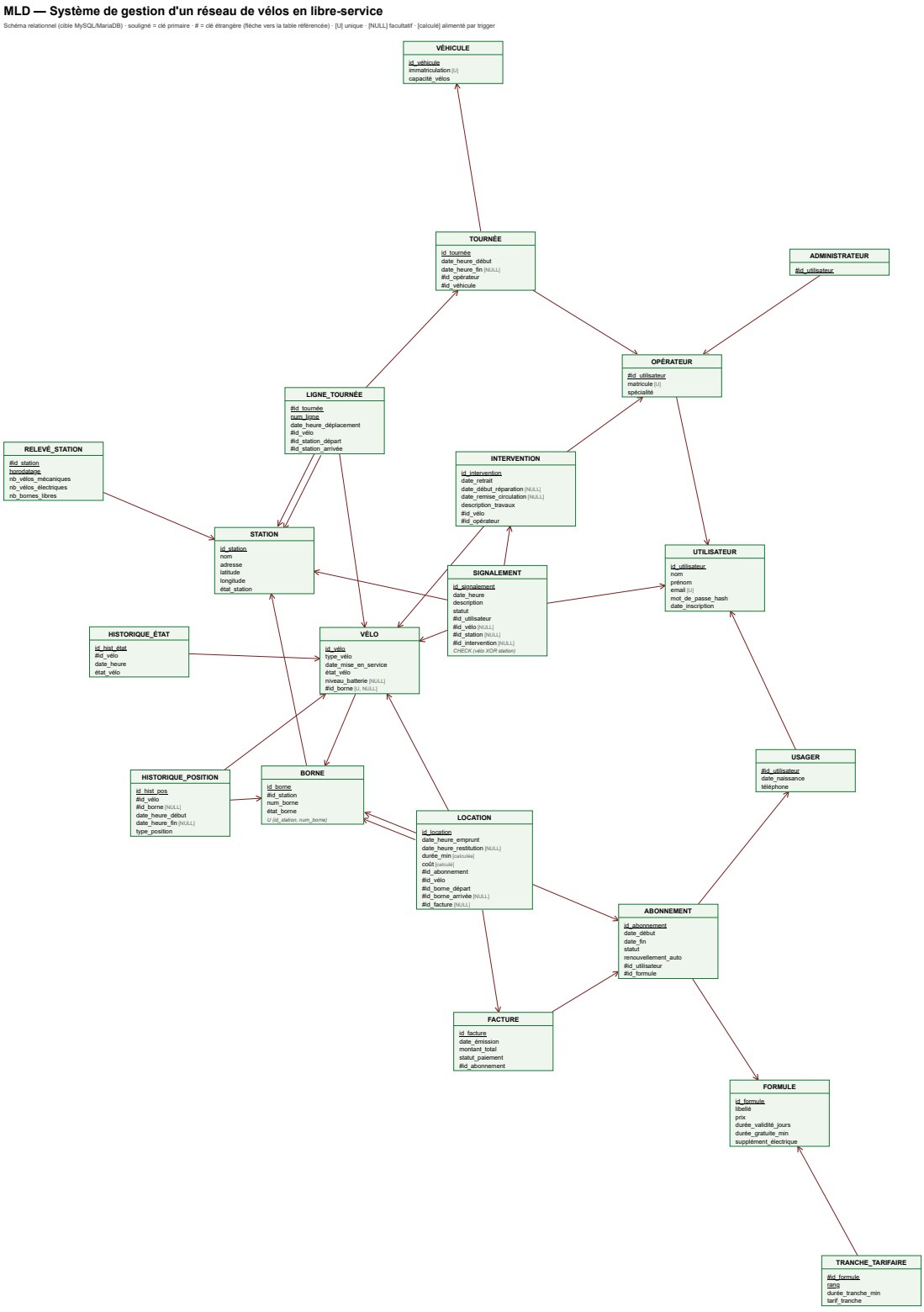


FIGURE 3 – Schéma relationnel (flèches : clés étrangères vers la table référencée)

Usager (#id_utilisateur, date_naissance, téléphone)
Opérateur (#id_utilisateur, matricule, spécialité)
Administrateur (#id_utilisateur)
Station (id_station, nom, adresse, latitude, longitude, état_station)
Borne (id_borne, #id_station, num_borne, état_borne)
Relevé_station (#id_station, horodatage, nb_vélos_mécaniques, nb_vélos_électriques, nb_bornes_libres)
Vélo (id_vélo, type_vélo, date_mise_en_service, état_vélo, niveau_batterie, #id_borne)
Historique_état (id_hist_état, #id_vélo, date_heure, état_vélo)
Historique_position (id_hist_pos, #id_vélo, #id_borne, date_heure_début, date_heure_fin, type_position)
Formule (id_formule, libellé, prix, durée_validité_jours, durée_gratuite_min, supplément_électrique)
Tranche_tarifaire (#id_formule, rang, durée_tranche_min, tarif_tranche)
Abonnement (id_abonnement, date_début, date_fin, statut, renouvellement_auto, #id_utilisateur, #id_formule)
Location (id_location, date_heure_emprunt, date_heure_restitution, durée_min, coût, #id_abonnement, #id_vélo, #id_borne_départ, #id_borne_arrivée, #id_facture)
Facture (id_facture, date_émission, montant_total, statut_paiement, #id_abonnement)
Signalement (id_signalement, date_heure, description, statut, #id_utilisateur, #id_vélo, #id_station, #id_intervention)
Intervention (id_intervention, date_retrait, date_début_réparation, date_remise_circulation, description_travaux, #id_vélo, #id_opérateur)
Tournée (id_tournée, date_heure_début, date_heure_fin, #id_opérateur, #id_véhicule)
Véhicule (id_véhicule, immatriculation, capacité_vélos)
Ligne_tournée (#id_tournée, num_ligne, date_heure_déplacement, #id_vélo, #id_station_départ, #id_station_arrivée)

Le passage du modèle conceptuel au modèle relationnel repose sur les choix suivants :

- **héritage par tables liées** : `usager` et `operateur` ont pour clé primaire la clé étrangère `id_utilisateur` ; `administrateur` référence `operateur`. Les attributs communs ne sont stockés qu'une fois, et chaque rôle n'a que ses propres colonnes ;
- **identifiant technique pour la borne** : l'identifiant relatif du modèle conceptuel (station, numéro) devient une contrainte d'unicité `(id_station, num_borne)`, et la borne reçoit une clé `id_borne` plus simple à référencer depuis les vélos, les locations et les historiques ;
- **clés composées pour les entités faibles sans dépendants** : `tranche_tarifaire(id_formule, rang)`, `releve_station(id_station, horodatage)` et `ligne_tournee(id_tournee, num_ligne)` ;
- **associations 0,1** : « stationner » devient la clé étrangère *unique* `id_borne` de `velo` ; la facturation d'un trajet devient la clé étrangère facultative `id_facture` de `location` ;
- la location référence ses **bornes** de départ et d'arrivée, et non les stations, qui s'en déduisent par jointure.

3.3 Normalisation

Le schéma respecte la troisième forme normale : chaque table décrit une seule entité ou association, tout attribut non clé dépend de toute la clé et de rien d'autre que la clé. Quelques attributs dérivables ont été conservés volontairement ; ils sont calculés par la base, jamais saisis, ce qui écarte les anomalies de mise à jour que la normalisation vise à éviter :

- `location.duree_min` est une **colonne générée** (`GENERATED ALWAYS AS ... STORED`) à partir des dates d'emprunt et de restitution : elle ne peut pas diverger de ces dates, et elle est indexable pour les statistiques ;

– `location.cout` est calculé par déclencheur à la restitution et **figé** : il dépend de la tarification *au moment du trajet*, qui peut évoluer ensuite ; le recalculer fausserait les factures passées ;
– `location.velo_en_cours` et `abo_en_cours` sont des colonnes générées, non nulles seulement pendant une location : un index `UNIQUE` sur chacune garantit « au plus une location en cours » par vélo et par abonnement (MariaDB n'offre pas d'index unique partiel) ;
– `abonnement.statut` : la résiliation n'est pas déductible des dates, et l'état « expiré » est matérialisé par un événement planifié pour permettre un accès indexé aux abonnements actifs.

3.4 Contraintes d'intégrité

Les règles exprimables de façon déclarative sont confiées au schéma, les autres aux déclencheurs (partie 4). Le tableau 2 en donne les principales.

Contrainte	Règle garantie
`uq_velo_borne (id_borne)`	une borne accueille au plus un vélo
`ck_velo_position`	un vélo en location ou en atelier n'est sur aucune borne : il ne peut être qu'à un seul endroit
`ck_velo_batterie_electrique`, `ck_velo_batterie_plage`	niveau de batterie réservé aux vélos électriques, entre 0 et 100
`uq_location_velo_en_cours`, `uq_location_abo_en_cours`	au plus une location en cours par vélo et par abonnement
`ck_loc_dates`, `ck_loc_restitution`	la restitution suit l'emprunt ; une location est terminée si et seulement si sa borne d'arrivée est connue
`ck_sig_cible`	un signalement vise soit un vélo, soit une station, jamais les deux
`ck_station_latitude`, `ck_station_longitude`	coordonnées géographiques valides
`ck_abo_dates`, `ck_hist_pos_dates`, `ck_interv_*`, `ck_tournee_dates`	cohérence chronologique des périodes
`ck_formule_*`, `ck_tranche_*`, `ck_facture_montant`, `ck_loc_cout`	montants et durées positifs
`ck_ligne_stations`	un déplacement de régulation change de station

TABLE 2 – Principales contraintes déclaratives

Les clés étrangères sont en `ON DELETE RESTRICT` par défaut : supprimer une station, un vélo ou un abonnement qui a un historique est refusé, ce qui protège les données de facturation et les statistiques. La suppression en cascade est réservée aux lignes sans existence propre : rôles d'un utilisateur (`usager`, `operateur`, `administrateur`), relevés d'une station, tranches d'une formule et lignes d'une tournée. Une contrainte de MySQL a aussi guidé ces choix : une colonne utilisée dans un `CHECK` ne peut pas porter d'action `CASCADE` ou `SET NULL`.

3.5 Index

Chaque clé étrangère est couverte par un index, condition pour vérifier rapidement l'intégrité référentielle et réaliser les jointures. Ces index ont été composés de façon à servir aussi les requêtes de l'application :

- **recherche de l'abonnement valide** d'un usager à l'emprunt : `(id_utilisateur, statut, date_fin)` sur `abonnement` ;
- **historiques d'un vélo**, triés par date : `(id_velo, date_heure)` sur les historiques d'état et de position, les locations et les signalements ;
- **file des signalements** par statut et par date : `(statut, date_heure)` ;
- **expiration des abonnements** par l'événement planifié : `(statut, date_fin)` ;
- **factures par période** : `(date_emission)` et `(id_abonnement, date_emission)`.

L'ordre des colonnes suit la règle du préfixe gauche : la colonne filtrée par égalité vient en premier, la colonne triée ou filtrée par intervalle ensuite. Les index couvrants ajoutés pour les statistiques sont étudiés en partie 8.

4 Implémentation SQL

Les règles métier sont implémentées dans la base : l'application n'exécute aucune écriture directe sur les tables sensibles et appelle des procédures stockées, dont elle affiche tels quels les messages d'erreur levés par `SIGNAL`.

4.1 Déclencheurs et historisation

- `trg_velo_ai`, `trg_velo_au` : historisation de chaque changement d'état (`historique_etat`) et de position (`historique_position`) d'un vélo ; chaque changement de position ferme la ligne ouverte (date de fin renseignée) et en ouvre une nouvelle ;
- `trg_location_bi`, `trg_location_bu` : contrôle des règles d'emprunt (abonnement valide, un seul vélo à la fois, vélo disponible) et calcul automatique du coût à la restitution ;
- `trg_abonnement_bi` : un seul abonnement actif à la fois par usager ;
- `trg_signalement_ai` : un vélo signalé défectueux ne peut plus être emprunté.

Pendant le chargement massif des données factices, les procédures de génération positionnent la variable de session `@seed_mode` : les déclencheurs sont alors inactifs pour cette session uniquement, le générateur produisant lui-même des historiques cohérents. Cela évite plusieurs millions d'exécutions de déclencheurs.

Le déclencheur `trg_velo_au` illustre le principe. Il compare l'ancienne et la nouvelle ligne du vélo : un changement d'état ajoute une ligne à `historique_etat` ; un changement de position (borne, location ou atelier) ferme la période ouverte de `historique_position` et en ouvre une nouvelle. Aucune procédure n'a donc à penser à l'historique : il suffit de mettre à jour le vélo.

```
CREATE TRIGGER trg_velo_au AFTER UPDATE ON velo
FOR EACH ROW
BEGIN
    DECLARE v_ancienne VARCHAR(12);
    DECLARE v_nouvelle VARCHAR(12);

    IF COALESCE(@seed_mode, 0) = 0 THEN          -- inactif pendant le chargement massif
        IF NOT (OLD.etat_velo <=> NEW.etat_velo) THEN
            INSERT INTO historique_etat (id_velo, date_heure, etat_velo)
            VALUES (NEW.id_velo, NOW(), NEW.etat_velo);
        END IF;

        -- position : sur une borne, en location ou hors réseau (atelier)
        SET v_ancienne = IF(OLD.id_borne IS NOT NULL, 'borne',
```

```
                        IF(OLD.etat_velo = 'en_location', 'en_location', 'en_atelier'));
        SET v_nouvelle = IF(NEW.id_borne IS NOT NULL, 'borne',
                        IF(NEW.etat_velo = 'en_location', 'en_location', 'en_atelier'));

        IF NOT (OLD.id_borne <=> NEW.id_borne) OR v_ancienne <> v_nouvelle THEN
            UPDATE historique_position          -- ferme la période en cours
               SET date_heure_fin = NOW()
             WHERE id_velo = NEW.id_velo AND date_heure_fin IS NULL;
            INSERT INTO historique_position     -- ouvre la nouvelle
                   (id_velo, id_borne, date_heure_debut, type_position)
            VALUES (NEW.id_velo, NEW.id_borne, NOW(), v_nouvelle);
        END IF;
    END IF;
END
```

L'opérateur <=> (égalité sûre vis-à-vis de `NULL`) est indispensable : `id_borne` vaut `NULL` pendant une location, et une comparaison <> ordinaire renverrait `NULL`, donc ne détecterait pas le départ ou l'arrivée du vélo.

4.2 Procédures et fonctions métier

- locations : `sp_emprunter_velo`, `sp_rendre_velo` (transactions, verrouillage du vélo et de la borne) ;
- abonnements : `sp_souscrire_abonnement`, `sp_renouveler_abonnement`, `sp_resilier_abonnement`, `sp_expirer_abonnements` ;
- facturation : `fn_cout_formule`, `fn_calculer_cout` (durée gratuite, tranches, supplément électrique), `sp_facturer_locations` ;
- maintenance : `sp_signaler_probleme`, `sp_ouvrir_intervention`, `sp_demarrer_reparation`, `sp_cloturer_intervention` ;
- régulation : `sp_stations_a_reequilibrer`, `sp_proposer_deplacements`, `sp_demarrer_tournee`, `sp_deplacer_velo_tournee`, `sp_terminer_tournee` ;
- statistiques : six procédures `sp_stat_*` (partie 8).

Le coût d'un trajet suit la formule de l'abonnement : les `duree_gratuite_min` premières minutes sont gratuites ; au-delà, les tranches s'appliquent dans l'ordre de leur rang, la dernière se répétant tant que le trajet continue (toute tranche commencée est due) ; le supplément électrique s'ajoute pour un vélo électrique. Par exemple, avec 30 minutes gratuites et deux tranches de 30 minutes à 1 € puis 2 €, un trajet de 100 minutes coûte $1 + 2 \times \lceil 40/30 \rceil = 5$ €.

Emprunt et restitution

L'emprunt (`sp_emprunter_velo`) et la restitution (`sp_rendre_velo`) sont les deux opérations exposées à la concurrence : deux usagers peuvent viser le même vélo ou la même borne au même instant. Chacune s'exécute dans une transaction unique et verrouille les lignes qu'elle lit avant de les modifier (`SELECT ... FOR UPDATE`) ; la seconde demande attend la fin de la première, puis constate que le vélo n'est plus disponible.

1. **Emprunt** : recherche et verrouillage de l'abonnement valide de l'usager, puis du vélo ; contrôle de son état (inconnu, défectueux, déjà en location ou en atelier), de sa présence sur une borne, et de l'état de la borne et de la station ; insertion de la location, puis passage du vélo à l'état « en location », sans borne. Les déclencheurs historisent le départ et revérifient les règles d'emprunt.

2. **Restitution** : verrouillage de la location en cours de l'usager et de la borne d'arrivée ; contrôle que la borne fonctionne, que la station est en service et qu'aucun vélo n'y est déjà posé ; enregistrement de la date de restitution, qui déclenche le calcul de la durée (colonne générée) et du coût (déclencheur) ; le vélo est reposé sur la borne, à l'état « disponible », ou « défectueux » si un signalement est encore ouvert à son sujet.

Toute anomalie lève une erreur `SIGNAL SQLSTATE '45000'` accompagnée d'un message métier (« Vélo signalé défectueux : emprunt impossible », « Cette borne est déjà occupée »…). Un gestionnaire `EXIT HANDLER FOR SQLEXCEPTION` annule alors la transaction (`ROLLBACK`) et propage l'erreur (`RESIGNAL`) ; l'application affiche ce message tel quel. Aucune opération ne peut ainsi laisser un vélo à moitié emprunté.

4.3 Événements planifiés

Trois événements automatisent les traitements périodiques :

- `ev_releve_stations`, toutes les 15 minutes : relevé de l'occupation des stations, qui alimente la statistique de remplissage ;
- `ev_expirer_abonnements`, toutes les heures : renouvellement automatique ou expiration des abonnements échus ;
- `ev_facturer_locations`, chaque nuit à 2 h : facturation des trajets payants.

Ils nécessitent l'activation de l'ordonnanceur (`event_scheduler=ON` dans la configuration du serveur). Cinq événements facultatifs, installés par `simulation.sql`, simulent en plus l'activité des usagers et des opérateurs (partie 5.3).

4.4 Sécurité

- l'application se connecte avec un compte dédié, `velo_app`, qui n'a que `SELECT` et `EXECUTE` sur la base, plus des droits d'écriture ciblés : insertion, modification et suppression sur `station` et `borne` ; modification de certaines *colonnes* seulement de `utilisateur`, `usager` et `signalement` ;
- mots de passe hachés avec bcrypt, régénération de la session à la connexion, limitation des tentatives ;
- jeton CSRF sur chaque formulaire, requêtes préparées uniquement, échappement automatique des gabarits, en-têtes HTTP de sécurité.

5 Jeu de données factices

5.1 Génération

Les données sont générées dans la base par les procédures `sp_seed_*` (`seed.sql`) : référentiel (formules, tranches), stations et bornes, flotte, utilisateurs, abonnements, locations, historiques, facturation, maintenance, régulation et relevés. Trois tailles sont proposées : `sp_seed_petit` (50 000 locations, pour les tests), `sp_seed_demo` (1 million) et `sp_seed_gros` (300 stations, 6 000 vélos, 500 000 usagers et 5 millions de locations), utilisée pour les mesures. La procédure `sp_seed_verifier` contrôle ensuite la cohérence du résultat.

Toutes les lignes sont produites par des requêtes ensemblistes (`INSERT ... SELECT`) par paquets, sans boucle ligne à ligne. Pendant le chargement, la variable de session `@seed_mode` désactive les déclencheurs : le générateur écrit lui-même des historiques cohérents. L'insertion massive en isolation `REPEATABLE READ` posait un verrou partagé sur chaque ligne lue et faisait déborder la table des verrous d'InnoDB (erreur 1206) : les procédures concernées passent en `READ COMMITTED`, où la lecture reste cohérente sans verrou.

5.2 Réalisme des données

Pour que les statistiques et les mesures de performances aient un sens, les données reproduisent les régularités d'un vrai réseau sur une année d'historique :

- **heures de pointe** : 27 % des trajets sont tirés autour de 8 h 15, 30 % autour de 17 h 45 et 13 % autour de midi (lois normales), le reste uniformément entre 6 h et 23 h ;
- **saisonnalité** : environ un tiers de trajets en moins le week-end et pendant les mois d'hiver ;
- **durées** : la plupart des trajets durent quelques minutes à une demi-heure (loi exponentielle de moyenne 14 min au-delà de 3 min), 3 % sont de longues locations d'une à onze heures, ce qui fait apparaître des coûts au-delà de la durée gratuite ;
- **géographie** : les stations sont réparties sur une zone d'environ 13 $\times$ 15 km ; 70 % des arrivées sont attirées vers les stations du centre, ce qui crée des stations saturées et des stations vides, et donc un besoin de régulation ;
- **continuité des vélos** : chaque vélo suit une chaîne de trajets sans chevauchement, chaque trajet partant de la borne où le précédent s'est terminé ;
- **clientèle** : 45 % d'abonnés annuels, 10 % au tarif réduit, 30 % d'abonnés mensuels (abonnements de 30 jours successifs) et 15 % d'usagers occasionnels, à qui un ticket journée est créé et facturé pour chaque jour d'utilisation ;
- **état courant** : environ 0,8 % des vélos sont en location, 1,2 % en atelier (intervention ouverte) et 2 % signalés défectueux ; les signalements, interventions et historiques passés sont cohérents avec ces états.

Quelques simplifications demeurent : la capacité des bornes n'est pas simulée dans le temps (seule la position actuelle respecte « une borne, un vélo »), et les déplacements des tournées de régulation ne sont pas reportés dans l'historique des positions.

5.3 Simulation de l'activité

Le jeu de données s'arrête au moment du chargement : sans activité, la carte, les tableaux de bord et les statistiques de la journée restent figés. Le script facultatif `simulation.sql` fait « vivre » la base en prolongeant l'historique en temps réel, par cinq événements planifiés (tableau 3).

Chaque événement appelle une procédure `sp_sim_*` qui joue le rôle des usagers et des opérateurs en appelant *les mêmes procédures métier que l'application* (`sp_emprunter_velo`, `sp_rendre_velo`, `sp_souscrire_abonnement`, `sp_ouvrir_intervention`, `sp_deplacer_velo_tournee`...). Les données simulées passent donc par les mêmes contrôles, déclencheurs (historisation, coût) et événements (relevés, expiration, facturation) que les actions réelles, et la simulation sert aussi de test continu des règles de gestion. Une action refusée par une règle (vélo emprunté entre-temps, borne occupée) est ignorée, comme le serait la tentative d'un vrai usager.

Événement	Fréquence	Activité simulée
`ev_sim_trajets`	1 min	restitutions puis emprunts, selon l'heure et le jour
`ev_sim_abonnements`	1 h	inscriptions de nouveaux usagers et réabonnements
`ev_sim_signalements`	15 min	signalements de vélos (qui passent en « défectueux ») et de stations
`ev_sim_maintenance`	15 min	retrait des vélos défectueux, réparation et remise en circulation, de 7 h à 19 h
`ev_sim_regulation`	1 h	tournées de rééquilibrage des stations pleines vers les plus vides, de 6 h à 22 h

TABLE 3 – Événements de la simulation

La simulation reprend les régularités du générateur :

- **profil horaire** : le nombre d'emprunts par minute suit un profil avec des pointes à 8 h et 18 h, un creux la nuit et une activité plus faible et plus étalée le week-end ; par défaut, 1 % de la flotte est emprunté chaque minute à l'heure de pointe ;
- **durées** : chaque minute, un trajet en cours se termine avec une probabilité de $1/15$ (durée moyenne de 15 minutes), et au plus tard au bout de 3 heures ;
- **mouvement pendulaire** : en semaine, la borne d'arrivée est choisie parmi trois bornes libres tirées au hasard, la plus centrale le matin et la plus éloignée du centre le soir ; les stations centrales se remplissent le matin et se vident le soir, ce qui donne du travail à la régulation.

Les tirages aléatoires partent d'un identifiant pris au hasard et lisent la clé primaire jusqu'au premier candidat valide, au lieu de trier toute la table (`ORDER BY RAND()`) : le coût d'un emprunt simulé ne dépend pas du volume de la base. Un verrou applicatif (`GET_LOCK`) empêche deux exécutions d'un même événement de se chevaucher, et la procédure `sp_sim_activer` suspend ou relance la simulation, par exemple pendant les mesures de performances.

Sur le petit jeu de données (300 vélos), une journée de semaine simulée en avançant l'horloge de session minute par minute s'exécute en une vingtaine de secondes : environ 1 400 trajets, une soixantaine d'abonnements, 24 tournées déplaçant 96 vélos. Les contrôles de `sp_seed_verifier` restent satisfaits, à une exception attendue : le contrôle de continuité des chaînes de trajets compte les vélos déplacés par une tournée ou reposés sur une autre borne après réparation, déplacements que le générateur ne produit pas mais qui sont normaux dans un réseau réel. Contrairement aux données générées, ces déplacements sont reportés dans l'historique des positions par les déclencheurs.

5.4 Volumes

Le tableau 4 donne les volumes de la base utilisée pour les mesures de performances ; il est lu dans `data/contexte.csv`, produit par le script de mesure.

Environnement		Volumes (lignes)	
Date de la mesure	2026-11-07	Locations	$4{,}99 \cdot 10^6$
Heure	13h29	Historique des états	$9{,}99 \cdot 10^6$
Fin des périodes	2026-11-07	Historique des positions	$9{,}99 \cdot 10^6$
Répétitions par mesure	5	Factures	$4{,}17 \cdot 10^6$
Serveur	10.4.32-MariaDB	Abonnements	$2{,}62 \cdot 10^6$
PHP	8.5.0	Relevés de stations	$2{,}6 \cdot 10^6$
Cache InnoDB	4 Go	Vélos	6 000
Taille de la base	4,7 Go	Stations	300

TABLE 4 – Contexte et volumes de la base mesurée

6 Application web

6.1 Architecture

L'application suit une architecture en couches autour du micro-framework Slim 4 :

- `public/index.php` : point d'entrée unique ;
- `config/` : paramètres (lus depuis `.env`), conteneur d'injection de dépendances, chaîne de middlewares (erreurs, en-têtes de sécurité, session, CSRF) et routes groupées par rôle ;
- `src/Controller/` : un contrôleur par domaine (stations, locations, abonnements, statistiques…) ;
- `src/Repository/` : toutes les requêtes SQL et les appels de procédures, ce qui isole l'accès aux données ;
- `templates/` : gabarits Twig (mise en page Tailwind, composants réutilisables).

La figure 4 suit une requête, par exemple l'emprunt d'un vélo. Les middlewares s'appliquent à toutes les routes ; le contrôle de rôle (`RequireRole`) est attaché à chaque groupe de routes : visiteur, utilisateur connecté, usager, opérateur, administrateur. Le contrôleur ne contient aucune requête SQL : il valide les paramètres, appelle le repository, qui exécute une requête préparée ou une procédure stockée, puis transmet le résultat au gabarit. Une erreur levée par la base (`SIGNAL`) remonte jusqu'au contrôleur, qui l'affiche comme message à l'utilisateur.

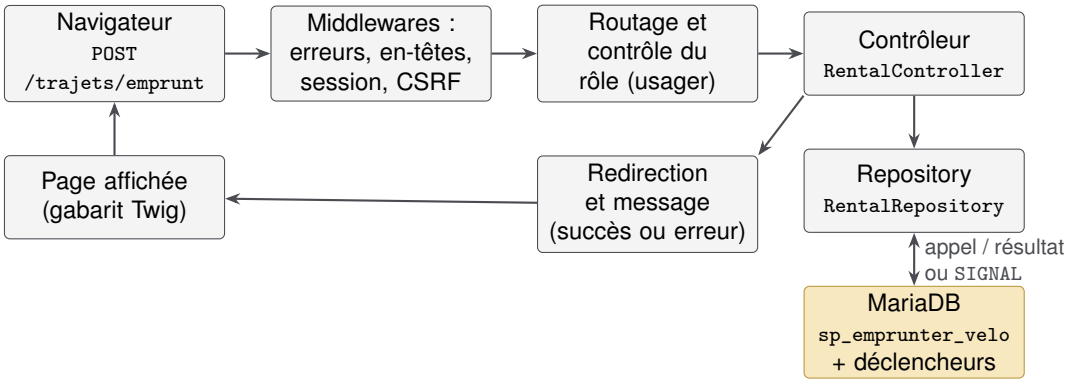


FIGURE 4 – Cheminement d'une requête dans l'application

6.2 Fonctionnalités par rôle

Le tableau 5 récapitule les pages de l'application et, pour chacune, le traitement réalisé par la base. Les fonctionnalités exigées par le sujet (inscription et connexion, stations en temps réel, gestion des stations, emprunt et restitution, statistiques) sont complétées par la gestion des abonnements et des factures, les signalements, le suivi de la flotte et des tableaux de bord adaptés à chaque rôle.

TABLE 5 – Fonctionnalités de l'application et traitements en base

Rôle	Fonctionnalité	Traitement en base
Visiteur	accueil avec la disponibilité du réseau ; inscription ; connexion (tentatives limitées)	vue `v_stations_occupation` ; `sp_inscrire_usager`, `sp_authentifier` ; mot de passe bcrypt vérifié par PHP
Usager	stations en temps réel (recherche, filtres)	vue `v_stations_occupation`
	souscription, renouvellement et résiliation d'un abonnement ; factures	`sp_souscrire_abonnement`, `sp_renouveler_abonnement`, `sp_resilier_abonnement`
	emprunt et restitution d'un vélo, avec le coût estimé en direct pendant le trajet	`sp_emprunter_velo`, `sp_rendre_velo` ; estimation par `fn_calculer_cout`
	historique des trajets ; signalement d'un problème ; profil	`sp_signaler_probleme` ; droits limités à certaines colonnes
Opérateur	tableau de bord d'exploitation ; toutes les locations (filtres par date, statut, station, type, usager)	index `(id_velo, date_heure_emprunt)`, `(id_abonnement, date_heure_emprunt)`
	flotte : filtres, fiche d'un vélo avec ses historiques d'état et de position	tables `historique_etat`, `historique_position`
	file des signalements : prise en charge, clôture	index `(statut, date_heure)`
	statistiques à onglets, graphiques, export CSV ; propositions de rééquilibrage	six procédures `sp_stat_*` ; `sp_stations_a_reequilibrer`, `sp_proposer_deplacements`
Administrateur	stations : création, modification, ajout et mise hors service de bornes, suppression	droits d'écriture de `velo_app` limités à `station` et `borne`
	comptes : recherche, création d'opérateurs	héritage `utilisateur` / `operateur` / `administrateur`

6.3 Page de statistiques

La page présente les six statistiques demandées, réparties en trois onglets (« Trajets », « Stations et régulation », « Recettes et flotte ») ; seules les procédures de l'onglet affiché sont exécutées. Les résultats sont mis en cache côté application (une heure pour une période passée, cinq minutes si elle inclut le jour courant, trente secondes pour le temps réel), et un lien permet de les recalculer pour mesurer le temps réel des procédures. Chaque rapport est exportable en CSV.

7 Évolution du projet

7.1 Chronologie

Le tableau 6 retrace les étapes du projet d'après l'historique du dépôt Git.

TABLE 6 – Principales étapes du projet

Date	Auteur	Étape
29/10/2026	C. Aubert	Prototype d'interface en PHP natif : pages de base, navigation, transitions, thème sombre.
31/10/2026	A. Prévost	Refonte de l'architecture du dépôt.
01/11/2026	A. Prévost	Diagrammes (cas d'utilisation, modèles conceptuel et logique) ; schéma SQL, contraintes et index ; déclencheurs et procédures ; premières procédures de génération de données.
05/11/2026	A. Prévost	Correction du chargement massif (débordement de la table des verrous InnoDB) ; compte applicatif à droits minimaux et comptes de démonstration.
05/11/2026	A. Prévost	Réécriture de l'application avec Slim 4 et Twig : authentification et rôles, stations, abonnements, locations, statistiques, signalements, flotte, tableaux de bord et gestion des comptes.
05/11/2026	A. Gratia	Schéma entité-association ; première optimisation de la page de statistiques : onglets, pagination, cache applicatif, index couvrants, réécriture des procédures de recettes et d'indisponibilité.
06/11/2026	A. Prévost	Tests sur la base complète (5 Go) : migration des index, configuration du serveur, réécriture des statistiques origine-destination, d'indisponibilité de la flotte et de remplissage ; banc de mesure automatisé.
07/11/2026	A. Prévost	Mesures finales et rédaction du rapport ; compilation automatisée (conversion des diagrammes SVG).
12/11/2026	A. Prévost	Mise à niveau des index intégrée à `procedures.sql` ; simulation de l'activité du réseau par événements planifiés.

Au-delà de l'historique, trois choix ont structuré le projet :

- **les règles métier dans la base** : contraintes, déclencheurs et procédures garantissent la cohérence quelle que soit l'application qui écrit, et l'application se contente d'afficher les erreurs levées ; ce choix a été fait dès le schéma SQL et n'a pas été remis en cause ;
- **des scripts ré-exécutables plutôt qu'une base partagée** : chaque membre reconstruit la base sur son poste à partir du dépôt, ce qui a imposé des scripts idempotents, puis une mise à jour en place des index quand la base est devenue trop volumineuse pour être recréée ;
- **un générateur de données intégré à la base** plutôt qu'un jeu de données importé : la volumétrie se règle par paramètres, et les mesures de performances peuvent être reproduites sur une base de même taille.

7.2 D'un prototype PHP natif à une application structurée

Le premier prototype mêlait requêtes SQL et affichage dans des pages PHP. Pour la version finale, l'application a été restructurée autour de Slim 4 : routes groupées par rôle, middlewares

de session, de CSRF et de contrôle d'accès, contrôleurs par domaine et repositories qui regroupent toutes les requêtes. Ce découpage rend chaque requête identifiable et mesurable, ce qui a facilité l'analyse des performances.

7.3 Problèmes rencontrés et solutions

Débordement des verrous lors du chargement massif. En isolation `REPEATABLE READ`, un `INSERT ... SELECT` pose un verrou partagé sur chaque ligne lue ; sur plusieurs millions de locations, la table des verrous d'InnoDB débordait (erreur 1206). Les procédures de génération passent en `READ COMMITTED`.

Différences entre MySQL et MariaDB. MariaDB signale une violation de contrainte `CHECK` par le code 4025 et non 3819 comme MySQL 8 : la couche d'accès aux données traduit les deux en message métier.

Droits du compte applicatif. Après une recréation de la base, le compte `velo_app` avait perdu son droit `SELECT, EXECUTE` sur la base : toutes les pages échouaient. Le script `bin/check-db.php` diagnostique désormais la connexion, les droits et la présence des données.

Page de statistiques bloquée sur la base complète. Avec le volume final, l'onglet affiché par défaut ne répondait plus. L'analyse, les corrections et les mesures font l'objet de la partie 8.

Mise à jour d'une base de plusieurs gigaoctets. Recréer la base pour modifier un index n'était plus envisageable. Les index modifiés ont d'abord été appliqués par un script de migration séparé, puis cette étape a été intégrée à `procedures.sql`, déjà ré-exécutable : il compare la définition de chaque index à celle du schéma (`information_schema.STATISTICS`) et ne le reconstruit que s'il diffère, sans bloquer la table (`ALGORITHM=INPLACE, LOCK=NONE`). Une base neuve et une base existante s'installent ainsi avec les mêmes scripts, sans liste de migrations à tenir à jour.

8 Analyse des performances

Cette partie constitue le rapport d'analyse des performances demandé par le sujet. Elle porte sur les requêtes les plus coûteuses du système, les six statistiques, exécutées sur la base complète, et retrace les mesures qui ont conduit à chaque correction.

8.1 Protocole de mesure

- **Base** : jeu de données complet (tableau 4), environ 5 Go.
- **Serveur** : MariaDB 10.4.32 (XAMPP) sous Windows 11, 32 Go de RAM, cache InnoDB de 4 Go, ordonnanceur d'événements actif (`event_scheduler=ON`) : un relevé des stations, planifié toutes les 15 minutes, peut ponctuellement s'exécuter pendant une mesure, ce que la médiane absorbe.
- **Procédures** : version finale de la branche `dev`, rechargée avant la mesure. Un premier passage avait été lancé par erreur sur les anciennes procédures : il redonnait les temps d'avant correction (415 s pour l'indisponibilité sur 30 jours), ce qui confirme au passage l'effet des réécritures.
- **Méthode** : chaque procédure est appelée directement, sans le cache de l'application, par le script `bin/bench-stats.php`. Pour chaque longueur de période (7, 30, 90 et 365 jours),

le premier appel est noté à part, car il inclut le chargement des pages en mémoire ; la valeur retenue est la **médiane** des appels suivants.

– **Exécution de l'application** : les temps de page sont mesurés avec `curl` sur le serveur PHP intégré, connecté en administrateur, avec et sans le cache applicatif.
– **Vérification** : chaque requête réécrite a été comparée ligne à ligne à l'ancienne version sur plusieurs périodes ; les résultats sont identiques.

La page de statistiques s'ouvre par défaut sur les 30 derniers jours : c'est la période de référence. L'objectif fixé est un affichage de chaque onglet en moins de 5 secondes sans cache.

8.2 Constat initial

Sur la base complète, l'onglet « Trajets », affiché par défaut, ne répondait plus : la requête HTTP restait en attente plus de deux minutes. La mesure de chaque procédure isolément a révélé trois requêtes problématiques : les trajets origine-destination (plus de 60 s), l'indisponibilité de la flotte (plus de 100 s) et, dans une moindre mesure, les recettes par formule (8,8 s). Deux causes, de natures différentes, se cumulaient.

8.3 Configuration du serveur

Le paramètre `innodb_buffer_pool_size`, mémoire qu'InnoDB consacre au cache des données et des index, valait 16 Mo, valeur par défaut de XAMPP, pour une base de près de 5 Go : presque chaque page lue l'était sur le disque. Il a été porté à 4 Go (et `innodb_log_file_size` à 512 Mo, environ un quart du cache). La figure 5 compare les procédures dont la version n'a pas changé entre les deux mesures.

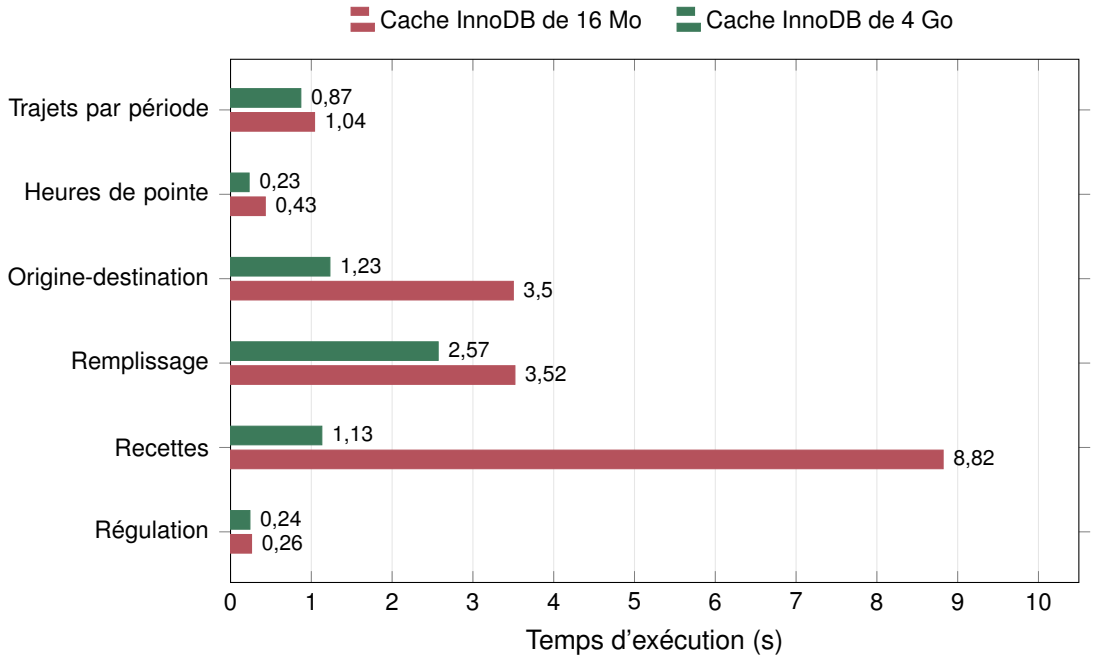


FIGURE 5 – Effet du cache InnoDB (période de 30 jours, mêmes versions des procédures)

Le gain est le plus fort pour les requêtes qui relisent beaucoup de lignes par clé primaire, comme les recettes (8,8 s → 1,1 s) : chaque facture de la période est relue dans la table, à des emplacements dispersés. Ce réglage ne suffisait toutefois pas : avec toutes les données en

mémoire, l'indisponibilité de la flotte prenait encore 22 s. Le problème restant tenait à la forme des requêtes.

8.4 Réécriture des requêtes

La figure 6 résume les gains obtenus par réécriture, chaque fois à configuration identique.

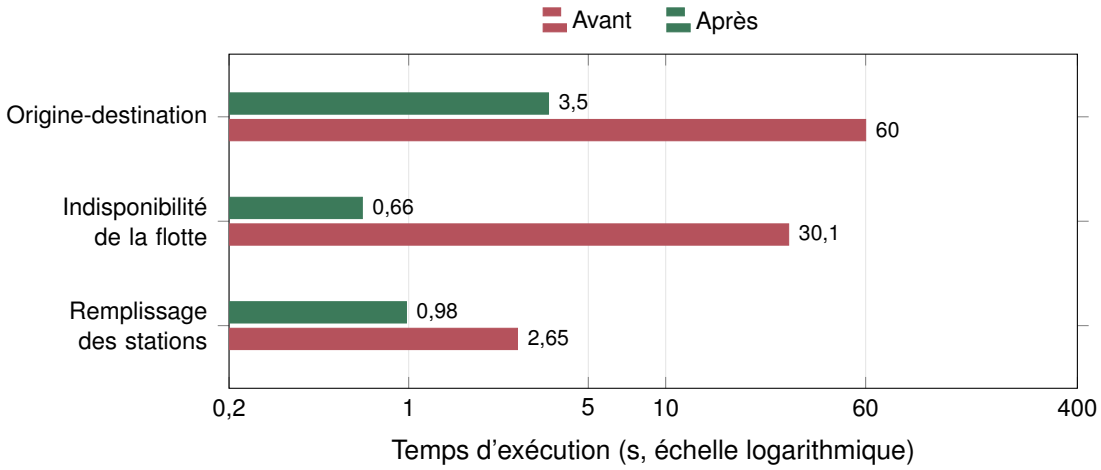


FIGURE 6 – Temps avant et après réécriture (30 jours). Origine-destination mesurée avec un cache de 16 Mo (avant : interrompue après 60 s) ; les deux autres avec 4 Go.

8.4.1 Trajets origine-destination

La requête initiale joignait `location` aux bornes et aux stations de départ et d'arrivée, puis filtrait sur la période. Son plan d'exécution (`EXPLAIN`) montre que l'optimiseur partait des stations, puis des bornes, puis des locations via l'index des arrivées :

```
table  type   key                   rows  Extra
sa     index  uq_station_nom         300  Using temporary; Using filesort
ba     ref    uq_borne_station_num    13  Using index
l      ref    idx_loc_arrivee_resti  243  Using where
```

Ce chemin lit *toutes* les locations de chaque borne (environ 5 millions) et ne filtre la date qu'ensuite, alors que la période n'en contient que 406 000. La version réécrite :

- impose l'ordre de jointure (`STRAIGHT_JOIN`) : la lecture part des locations de la période, par une plage sur l'index couvrant `idx_loc_emprunt`, sans relire la table ;
- regroupe par paire de *stations* (au plus 300^2 groupes) et non par paire de bornes (jusqu'à $7\,800^2$), ce qui garde la table temporaire en mémoire ; une première réécriture qui regroupait d'abord par bornes prenait encore 27 s ;
- ne joint les noms des stations qu'au résultat agrégé.

```
SELECT sd.id_station, sd.nom, sa.id_station, sa.nom,
       x.nb_trajets, ROUND(x.duree_moyenne_min, 1)
  FROM (SELECT STRAIGHT_JOIN bd.id_station AS id_depart, ba.id_station AS id_arrivee,
               COUNT(*) AS nb_trajets, AVG(l.duree_min) AS duree_moyenne_min
          FROM location l
          JOIN borne bd ON bd.id_borne = l.id_borne_depart
          JOIN borne ba ON ba.id_borne = l.id_borne_arrivee
         WHERE l.date_heure_emprunt >= p_debut AND l.date_heure_emprunt < p_fin
```

```
        GROUP BY bd.id_station, ba.id_station) x
  JOIN station sd ON sd.id_station = x.id_depart
  JOIN station sa ON sa.id_station = x.id_arrivee
 ORDER BY x.nb_trajets DESC, sd.nom, sa.nom
 LIMIT p_limite;
```

Le nouveau plan lit la plage de dates de l'index (`range`, `Using index`) puis accède aux bornes par clé primaire (`eq_ref`). Temps : plus de 60 s → 3,5 s avec le cache de 16 Mo, 1,2 s avec 4 Go.

8.4.2 Indisponibilité de la flotte

Le taux d'indisponibilité est la part du temps passée « en atelier » ou « défectueux », calculée à partir de `historique_etat`. La version initiale reconstituait *tous* les intervalles d'état de la période avec la fonction de fenêtre `LEAD` : sur 30 jours, 813 000 changements d'état, dont la plupart (« disponible » / « en location ») sont dus aux locations. La décomposition de la requête a montré deux coûts : la relecture de chaque ligne par clé primaire pour obtenir `id_velo` (6 s), absent de l'index utilisé, et la fonction de fenêtre elle-même (environ 4 s, même avec un index couvrant). Ajouter un index ne pouvait donc pas suffire.

Or seuls les intervalles d'indisponibilité comptent au numérateur : il n'y a que 174 passages en atelier ou défectueux sur 30 jours (788 sur un an). La procédure a été réécrite sans fonction de fenêtre :

- **numérateur** : pour chaque passage dans un état indisponible pendant la période, ainsi que pour l'état de chaque vélo à l'ouverture de la période, la fin de l'intervalle est l'événement suivant du même vélo, trouvé par une seule lecture de l'index `(id_velo, date_heure)` ;
- **dénominateur** : les intervalles d'un vélo se suivent sans interruption de son premier événement connu jusqu'à la fin de la période ; le temps observé vaut donc *fin* − max(*début*, *premier événement*), soit une lecture d'index par vélo.

Un dernier écueil est apparu : dans une sous-requête corrélée, MariaDB n'optimise pas `MIN(date_heure)` en une simple lecture de la première entrée d'index, et parcourait les quelque 1 600 entrées de chaque vélo (10 millions au total, 4,9 s). L'écriture équivalente `ORDER BY date_heure LIMIT 1` ne lit qu'une entrée.

Les résultats sont identiques à ceux de l'ancienne version sur 8 jours, 30 jours et un an ; le temps passe de 30 s à 0,7 s sur 30 jours et de 465 s à 6,5 s sur un an (figure 7).

8.4.3 Remplissage des stations

Même défaut que pour les trajets origine-destination : l'optimiseur partait des 300 stations et lisait par clé primaire *tous* les relevés de chacune (2,6 millions) avant de filtrer la date. Les relevés de la période (777 000) sont désormais agrégés d'abord, par une plage sur l'index couvrant `idx_releve_horodatage`, puis joints aux stations : 2,65 s → 0,98 s, sur les mêmes 7 200 lignes de résultat.

8.4.4 Index couvrants

Deux index ont été étendus pour que les statistiques ne lisent que l'index, sans relire la table :

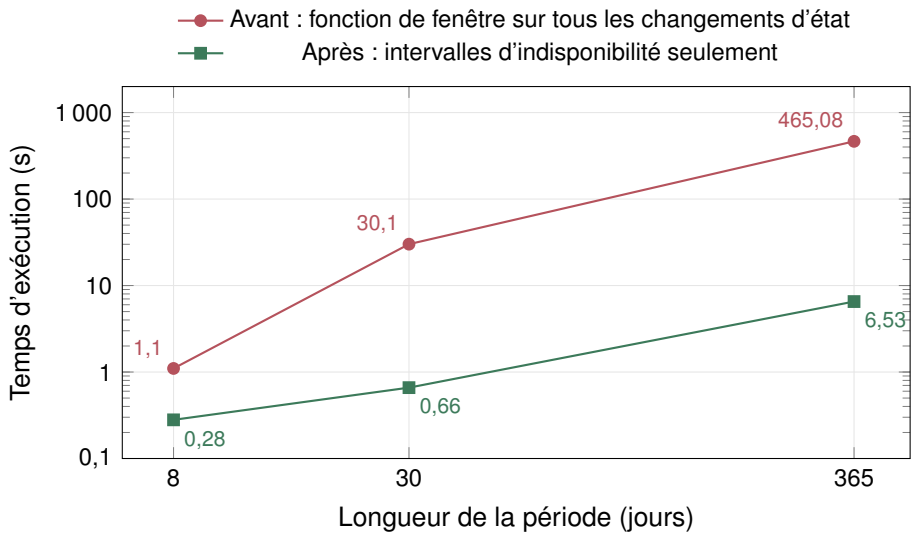


FIGURE 7 – Indisponibilité de la flotte selon la longueur de la période (cache de 4 Go)

- `location.idx_loc_emprunt` : `(date_heure_emprunt, date_heure_restitution, id_borne_depart, id_borne_arrivee, duree_min, cout)` ;
- `releve_station.idx_releve_horodatage` : `(horodatage, id_station, nb_velos_mecaniques, nb_velos_electriques, nb_bornes_libres)`.

La base existante a été mise à jour sans la recréer : `procedures.sql` reconstruit en place les index dont la définition diffère du schéma (partie 7).

8.5 Résultats

8.5.1 Temps des requêtes principales

Le tableau 7 donne le temps de chaque procédure sur la période de référence, après toutes les corrections.

Procédure	1er appel (s)	Médiane (s)	Min (s)	Max (s)	Lignes
Trajets par période	0,85	0,86	0,79	1,22	27
Heures de pointe	0,31	0,26	0,24	0,35	19
Trajets origine-destination	2,13	1,96	1,87	2,32	500
Remplissage des stations	1,14	1,07	0,97	1,17	7 200
Recettes par formule	1,35	1,55	1,46	1,59	4
Indisponibilité de la flotte	0,54	0,59	0,56	0,61	3
Stations à rééquilibrer	0,07	0,05	0,05	0,07	113
Déplacements proposés	0,11	0,14	0,11	0,18	0

TABLE 7 – Temps d'exécution des procédures statistiques sur 30 jours

Les écarts avec les temps cités plus haut (origine-destination : 1,2 s lors de la mise au point, 2,0 s ici) tiennent à la variabilité d'une machine de bureau ; les ordres de grandeur sont stables.

Chaque onglet de la page n'exécute que ses propres procédures, plus les indicateurs clés (trajets par période et heures de pointe) affichés en tête de chaque onglet. En additionnant les médianes, on obtient le temps de calcul de chaque onglet (tableau 8).

Onglet	Procédures appelées	30 j	90 j	365 j
Trajets	indicateurs + origine-destination	3,1 s	9,7 s	30,6 s
Stations	indicateurs + remplissage + rééquilibrage + déplacements	2,4 s	7,4 s	17,5 s
Recettes	indicateurs + recettes + indisponibilité	3,3 s	8,8 s	28,0 s

TABLE 8 – Temps de calcul de chaque onglet, sans cache (somme des médianes)

Sur la période de référence de 30 jours, les trois onglets respectent l'objectif de 5 s, avec une marge de 35 à 50 %. Au-delà de trois mois, l'objectif n'est plus tenu sans cache (voir les limites ci-dessous).

8.5.2 Montée en charge

La figure 8 montre l'évolution du temps de chaque statistique avec la longueur de la période.

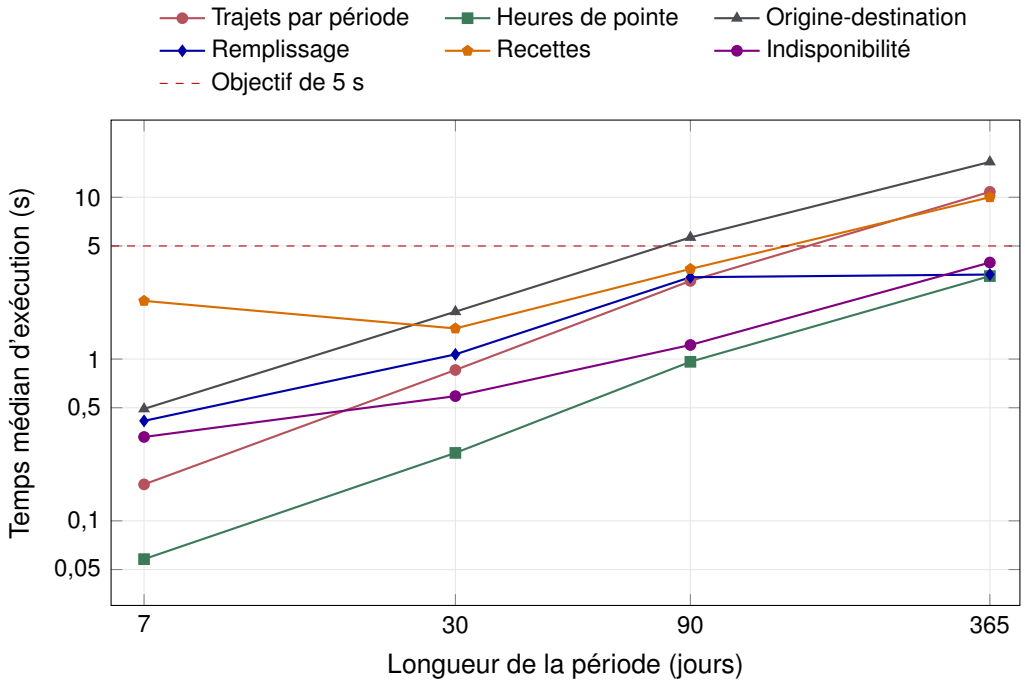


FIGURE 8 – Temps médian de chaque statistique selon la longueur de la période

Le temps croît à peu près linéairement avec le nombre de lignes de la période. Les procédures lisent désormais uniquement les lignes utiles, par des plages d'index : leur coût suit donc la taille de la période, et non plus celle de la table entière.

8.5.3 Temps d'affichage et cache applicatif

Mesurés de bout en bout par HTTP, avec le lien « Recalculer sans le cache », les onglets répondent en 3,0 s (Trajets), 2,5 s (Stations) et 3,4 s (Recettes) : le rendu PHP et Twig n'ajoute que quelques dixièmes de seconde au calcul des procédures. Une fois les résultats en cache, chaque onglet s'affiche en moins de 0,1 s, quelle que soit la période (figure 9). Lors d'une mesure intermédiaire, avec la requête origine-destination déjà réécrite mais pas encore le

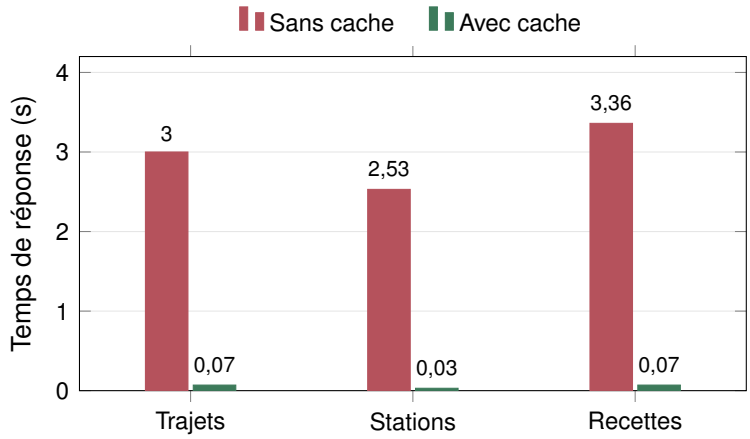


FIGURE 9 – Temps de réponse de chaque onglet de la page de statistiques (30 jours, médiane de 3 requêtes)

remplissage, l'onglet Stations prenait 5,1 s : c'est ce dépassement qui a motivé la dernière réécriture.

8.6 Limites et pistes d'amélioration

- Sur une période d'un an, les statistiques de trajets agrègent l'essentiel des 5 millions de locations : sans cache, les onglets prennent alors de 17 à 31 s (tableau 8), la requête origine-destination à elle seule 16,5 s. Le cache d'une heure sur les périodes passées en atténue l'effet, mais le premier calcul reste long.
- La piste naturelle est une **table d'agrégats journaliers** (trajets par jour et par paire de stations, recettes par jour et par formule), alimentée chaque nuit par un événement planifié : les statistiques liraient quelques milliers de lignes au lieu de plusieurs millions, au prix d'une dénormalisation contrôlée.
- Le partitionnement des grandes tables par date (`location`, `historique_etat`) limiterait aussi les lectures aux partitions de la période.
- Les performances dépendent fortement de la configuration du serveur : avec le cache InnoDB par défaut de 16 Mo, aucune réécriture ne suffit à tenir l'objectif.

9 Conclusion

Le système réalisé couvre l'ensemble du cahier des charges : la base de données modélise toutes les fonctionnalités demandées et en garantit les règles métier par des contraintes, des déclencheurs et des procédures stockées, et l'application web dépasse le minimum exigé en exposant, en plus de l'authentification, des stations, des locations et des statistiques, la gestion des abonnements, des signalements, de la flotte et des comptes.

Le passage à un jeu de données réaliste de plusieurs gigaoctets a été l'étape la plus formatrice : il a révélé des requêtes correctes mais inexploitables à ce volume, et imposé une démarche rigoureuse de mesure, de lecture des plans d'exécution et de réécriture, en vérifiant à chaque fois que les résultats restaient identiques.

Pour l'équipe, ce projet a surtout montré l'intérêt de confier les règles métier à la base : une fois les contraintes, les déclencheurs et les procédures en place, l'application a pu être réécrite entièrement (du prototype PHP natif à Slim) sans jamais remettre en cause la cohérence des

données. Il a aussi montré qu'une requête correcte sur quelques milliers de lignes peut devenir inutilisable sur plusieurs millions, et que seule la mesure permet de trouver où agir : la configuration du serveur, la forme des requêtes et les index ont chacun eu un effet propre.

Le système garde des limites :

- sans cache, les statistiques sur plus de trois mois dépassent l'objectif de 5 secondes (de 17 à 31 s sur un an) ;
- la maintenance (interventions) et la régulation (tournées) sont complètes dans la base mais seulement en partie exposées dans l'interface : file des signalements et propositions de rééquilibrage ;
- certaines simplifications du jeu de données (capacité des bornes non simulée dans le temps, tournées absentes de l'historique des positions) limitent le réalisme de quelques statistiques.

Plusieurs prolongements en découlent naturellement :

- des **tables d'agrégats journaliers** alimentées chaque nuit par un événement planifié, pour des statistiques sur un an en quelques dixièmes de seconde ;
- un **partitionnement par date** des tables d'historique et de locations ;
- des écrans opérateur pour les interventions et les tournées, s'appuyant sur les procédures existantes ;
- une **carte des stations** en temps réel, les coordonnées étant déjà stockées et indexées.

A Installation

Les scripts SQL s'exécutent dans cet ordre avec le compte `root` : `schema.sql`, `triggers.sql`, `procedures.sql`, `seed.sql` (puis `CALL sp_seed_petit();`), `app_user.sql`, `demo_accounts.sql` et, facultativement, `simulation.sql`. Une base existante se met à jour en ré-exécutant `procedures.sql`, qui reconstruit au passage les index dont la définition a changé.

Configuration recommandée du serveur (`my.ini`, section `[mysqld]`) :

```
event_scheduler=ON              -- événements planifiés
innodb_buffer_pool_size=4G      -- XAMPP fournit 16M
innodb_log_file_size=512M       -- environ 25 % du buffer pool
```

L'application s'installe avec `composer install`, se configure dans `.env` (mot de passe du compte `velo_app`) et se lance avec `composer start`. La commande `composer check-db` vérifie la connexion, les droits et la présence des données.

B Reproduire les mesures

```
php bin/bench-stats.php --repetitions=5 --periodes=7,30,90,365
```

Le script écrit trois fichiers dans `var/bench/` : `procedures.csv` (une ligne par procédure et par période), `echelle.csv` (les mêmes temps médians, une colonne par procédure) et `contexte.csv` (serveur et volumes). Copiés dans `docs/report/data/`, ils mettent à jour automatiquement les tableaux 4 et 7 ainsi que la figure 8 à la compilation suivante. Les procédures mesurées doivent être celles de la branche `dev` : rechargez `procedures.sql` depuis cette

branche avant la mesure, et suspendez la simulation (`CALL sp_sim_activer(FALSE);`) si elle est installée.

Le rapport se compile avec `docs/report/compiler.ps1`, qui convertit d'abord les diagrammes SVG en PDF.